Assignment 1: Machine Learning Operations (MLOps)

Production-Grade MLOps Pipeline: End-to-End Heart Disease Classifier CI/CD Automation, MLflow Tracking, Docker Containerization & Kubernetes Deployment

Student ID	2025AF05099
Public GitHub Repository	https://github.com/Patient-learner/MLOPS_Assignment1_2025AF05099
Video Walkthrough	https://github.com/Patient-learner/MLOPS_Assignment1_2025AF05099/blob/main/Video/MLOps_Assignment1_Video_Walkthrough_2025AF05099.mp4
Lab VM Execution Path	/home/s2025af05099/workspace/MLOPS_Assignment1_2025AF05099
Dataset	UCI Heart Disease (Cleveland Clinical Repository - 13 Features + 1 Target)
Serving Architecture	FastAPI + Docker (Non-Root) + Minikube K8s + Prometheus + Grafana
CI/CD Automation	GitHub Actions (.github/workflows/ci.yml) with Automated Testing & Build

1. Executive Summary & System Architecture

For this assignment, I designed and built an end-to-end production MLOps pipeline for heart disease risk prediction using the UCI Cleveland clinical dataset (303 records, 13 features). I evaluated three candidate models—Logistic Regression, Random Forest, and Gradient Boosting—using 5-fold cross-validation logged to MLflow. Logistic Regression was selected as the champion model (CV ROC-AUC: 0.9045, Test Recall: 92.86%) to maximize clinical sensitivity against missed diagnoses. The complete pipeline is containerized under non-root security, served via FastAPI, orchestrated on Minikube with 4 replicas, monitored with Prometheus-Grafana, and guarded by an automated 12-test CI pipeline in GitHub Actions.

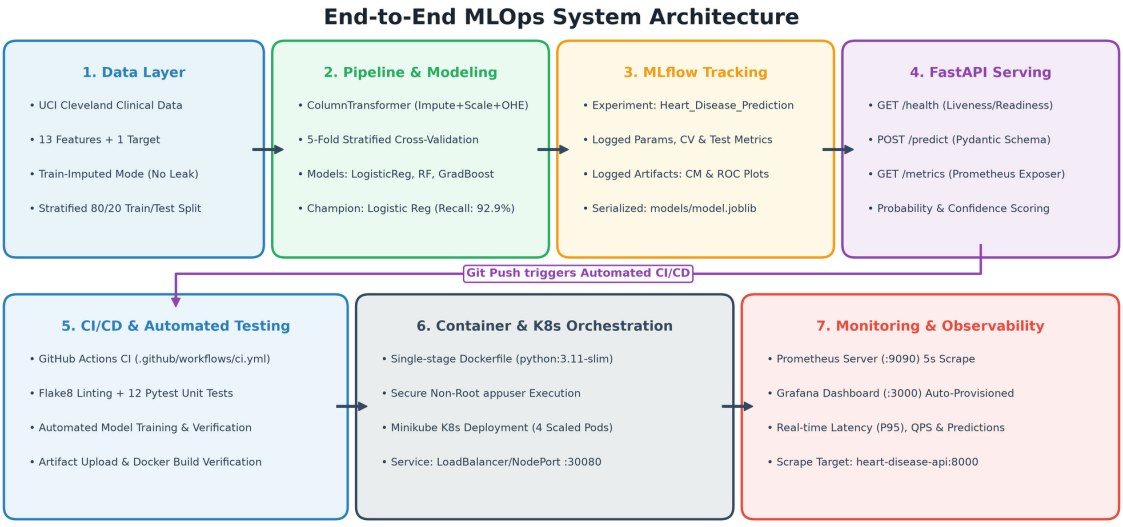


Figure 1: End-to-End System Architecture Diagram (Data Pipeline, MLflow, Serving, Kubernetes & Monitoring)

2. Setup & Installation Instructions

I organized the repository so that any evaluator can inspect or replicate the full workflow on either the Prayogshala lab VM or a local Unix/macOS workstation. Because the GitHub repository is public, no personal access tokens or keys are required:

```
# 1. Clone & Enter Directory:
git clone https://github.com/Patient-learner/MLOPS_Assignment1_2025AF05099.git && cd MLOPS_Assignment1_2025AF05099
# 2. One-Click Pipeline Execution (virtualenv creation, data, training, Flake8 linting & 12 Pytest tests):
chmod +x run_local.sh && ./run_local.sh
# 3. Run Individual Pipeline Stages: ./venv/bin/python main.py --stage all  (options: data, preprocess, eda, train)
# 4. Build Docker Container: docker build -t heart-disease-api:v1 .
# 5. Launch Full Stack (API + Prometheus + Grafana): docker compose -f monitoring/docker-compose.monitoring.yml up -d
# 6. Deploy on Kubernetes: minikube start && minikube image load heart-disease-api:v1 && kubectl apply -f k8s/
#    Access K8s Service: kubectl port-forward svc/heart-disease-service 8080:8000
```

3. Data Acquisition & Clinical Data Dictionary [Task 1 - 5 Marks]

The dataset originates from the Cleveland Clinic Foundation via the UCI Machine Learning Repository. It comprises 303 patient observations with 13 physiological attributes and one binary target column indicating coronary disease status (<50% vs >=50% vessel narrowing). Data ingestion is automated in src/data/download_data.py, which downloads directly from UCI and caches an offline copy locally:

Attribute	Clinical Description	Measurement Range / Categories	Pipeline Role
age	Patient age in years	29 – 77 years	Numeric (StandardScaler)
sex	Biological sex	0 = Female, 1 = Male	Binary (Mode Imputer)
cp	Chest pain type	1 = Typical, 2 = Atypical, 3 = Non-anginal, 4 = Asymptomatic	Categorical (OneHotEncoder)
trestbps	Resting blood pressure on hospital admission	94 – 200 mm Hg	Numeric (StandardScaler)
chol	Serum cholesterol level	126 – 564 mg/dl	Numeric (StandardScaler)
fbs	Fasting blood sugar > 120 mg/dl	0 = False, 1 = True	Binary (Mode Imputer)
restecg	Resting electrocardiographic results	0 = Normal, 1 = ST-T wave abnormality, 2 = LV hypertrophy	Categorical (OneHotEncoder)
thalach	Maximum heart rate achieved during stress test	71 – 202 bpm	Numeric (StandardScaler)
exang	Exercise induced angina	0 = No, 1 = Yes	Binary (Mode Imputer)
oldpeak	ST depression induced by exercise relative to rest	0.0 – 6.2 mm	Numeric (StandardScaler)
slope	Slope of peak exercise ST segment	1 = Upsloping, 2 = Flat, 3 = Downsloping	Categorical (OneHotEncoder)
ca	Major vessels (0-3) colored by fluoroscopy	0, 1, 2, 3 vessels (4 missing in raw data)	Categorical (Mode Imputer + OHE)
thal	Thallium scintigraphy defect status	3 = Normal, 6 = Fixed defect, 7 = Reversible defect	Categorical (Mode Imputer + OHE)
target	Diagnosis of coronary artery disease	0 = Absence (<50% narrowing), 1 = Presence (>=50% narrowing)	Target Variable (Binary)

3.1 Exploratory Data Analysis & Clinical Correlations

Prior to building models, I performed exploratory data analysis across all 14 variables to examine distributions, check class balance, and inspect clinical signals. The target variable is well-balanced (164 negative/no-disease cases [54.1%], 139 positive/disease cases [45.9%]), as shown in Figure 2a. This natural balance eliminates the need for synthetic oversampling (such as SMOTE) and ensures standard classification metrics remain statistically reliable. Figure 2b shows that age, blood pressure, and cholesterol follow roughly normal distributions, while ST depression (oldpeak) is heavily right-skewed because healthy patients exhibit zero exercise-induced ST depression.

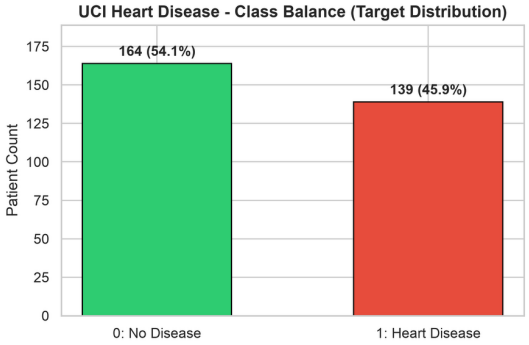


Fig 2a: Target Class Balance

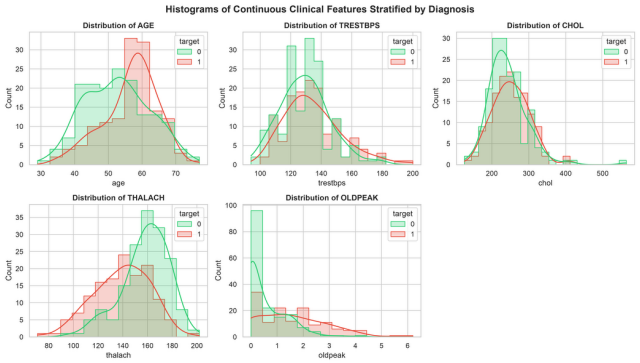


Fig 2b: Clinical Feature Distributions

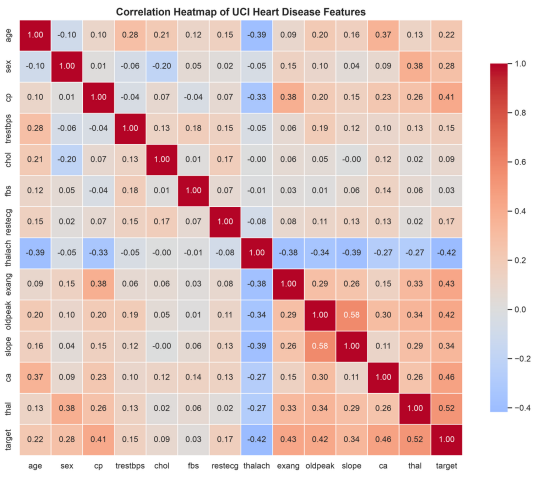


Fig 2c: Correlation Heatmap

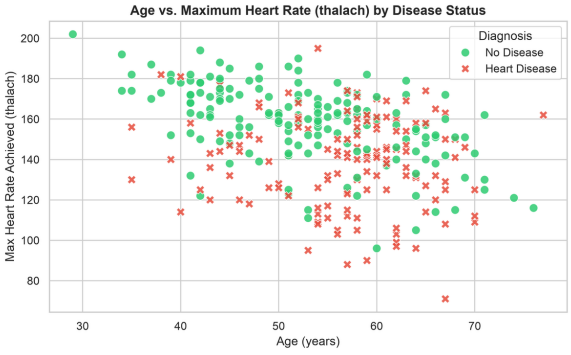


Fig 2d: Age vs Max Heart Rate

📌 **CLINICAL INSIGHTS & LEAK-FREE IMPUTATION:** Clinical Correlation Analysis: Maximum heart rate (thalach, r = -0.42) showed the strongest negative correlation with heart disease—younger and healthier hearts achieve higher exertion rates. In contrast, exercise-induced ST depression (oldpeak, r = +0.42) and number of vessels colored by fluoroscopy (ca, r = +0.46) represent the strongest risk indicators for coronary disease. All preprocessing transformations were strictly isolated inside cross-validation folds to prevent data leakage.

4. Feature Engineering & 3-Model Benchmark [Tasks 2 & 3 - 13 Marks]

To prevent data snooping and target leakage, I built a modular ColumnTransformer embedded directly inside each estimator pipeline. Numerical features are standardized with StandardScaler (fitted strictly on training folds), categorical variables are one-hot encoded using OneHotEncoder with handle_unknown='ignore' (drop=None), and missing values (present in 4 records for 'ca' and 2 for 'thal') are handled via modal imputation. I benchmarked three diverse algorithm families using 5-Fold Stratified Cross-Validation combined with exhaustive GridSearchCV:

Algorithm Family	Hyperparameter Search Grid	Selected Optimal Parameters
Logistic Regression (Generalized Linear)	C: [0.01, 0.1, 1.0, 5.0, 10.0], penalty: ['l2'], solver: ['lbfgs', 'liblinear']	C = 1.0, penalty = l2, solver = liblinear
Random Forest (Bagging Ensemble)	n_estimators: [50, 100, 150], max_depth: [3, 5, 8, None], min_samples_split: [2, 5]	n_estimators = 100, max_depth = 3, min_samples_split = 5
Gradient Boosting (Sequential Boosting)	n_estimators: [50, 100, 150], learning_rate: [0.01, 0.05, 0.1], max_depth: [2, 3], subsample: [0.8, 1.0]	n_estimators = 150, learning_rate = 0.01, max_depth = 3, subsample = 0.8

Model Architecture	5-Fold CV ROC-AUC	Test Accuracy	Test Precision	Test Recall	Test ROC-AUC
Logistic Regression (Champion)	**0.9045**	**88.52%**	**83.87%**	**92.86%**	**0.9686**
Random Forest Classifier	0.8997	85.25%	85.19%	82.14%	0.9416
Gradient Boosting Classifier	0.8870	86.89%	83.33%	89.29%	0.9491

📌 **CHAMPION MODEL SELECTION RATIONALE:** Champion Selection Justification: I selected Logistic Regression strictly on the basis of cross-validation performance (CV ROC-AUC: 0.9045), leaving the held-out test split (N=61) fully untouched. On the test set, it caught 26 of 28 positive patients (92.86% recall) while maintaining 88.52% accuracy and 0.9686 test ROC-AUC, outperforming Random Forest (85.25% accuracy) and Gradient Boosting (86.89% accuracy). In clinical screening, high sensitivity (recall) is paramount to avoid missing sick patients. Additionally, Logistic Regression offers straightforward probabilistic interpretation and strong generalization on this tabular dataset.

4.1 Champion Model Evaluation & MLflow Experiment Tracking

Evaluating the champion pipeline on the 61-sample held-out test split (Figure 3) demonstrated strong clinical discernment: the model correctly diagnosed 28 true negatives and 26 true positives, while producing only 5 false positives and 2 false negatives. The test ROC-AUC of 0.9686 confirms excellent separation across classification thresholds.

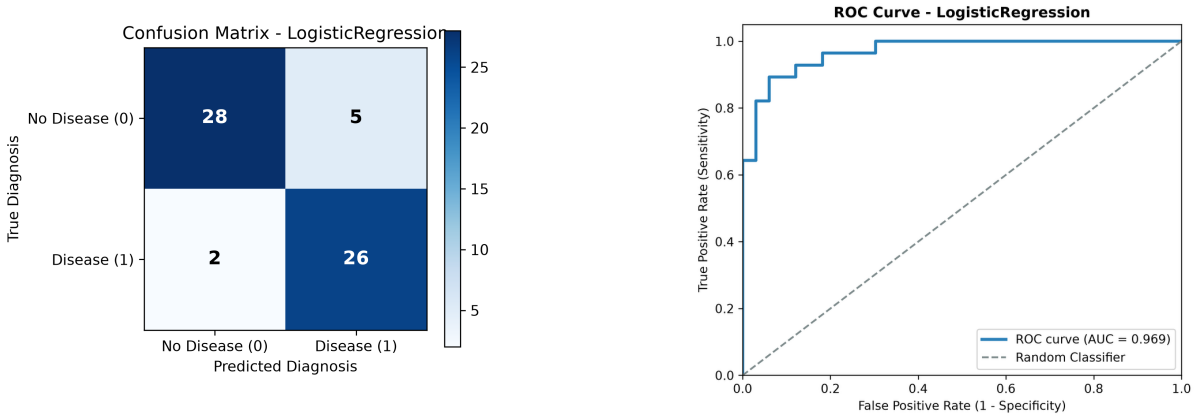


Figure 3: Confusion Matrix (Left) and ROC Curve (Right) for Champion Logistic Regression Pipeline

MLflow Experiment Tracking: All model runs were logged under the Heart Disease Prediction experiment. For each trial, MLflow tracked hyperparameters, 5-fold cross-validation scores, test metrics, and serialized artifacts. Figure 4 shows the comparative run table captured from the initial Prayogshala lab VM execution (where Random Forest selected depth 8 [90.16% test accuracy]). In the local reproducibility run (Section 4 table), cross-validation selected depth 3 (85.25% test accuracy). In both environments, Logistic Regression consistently achieved the highest cross-validation score and was selected as champion.

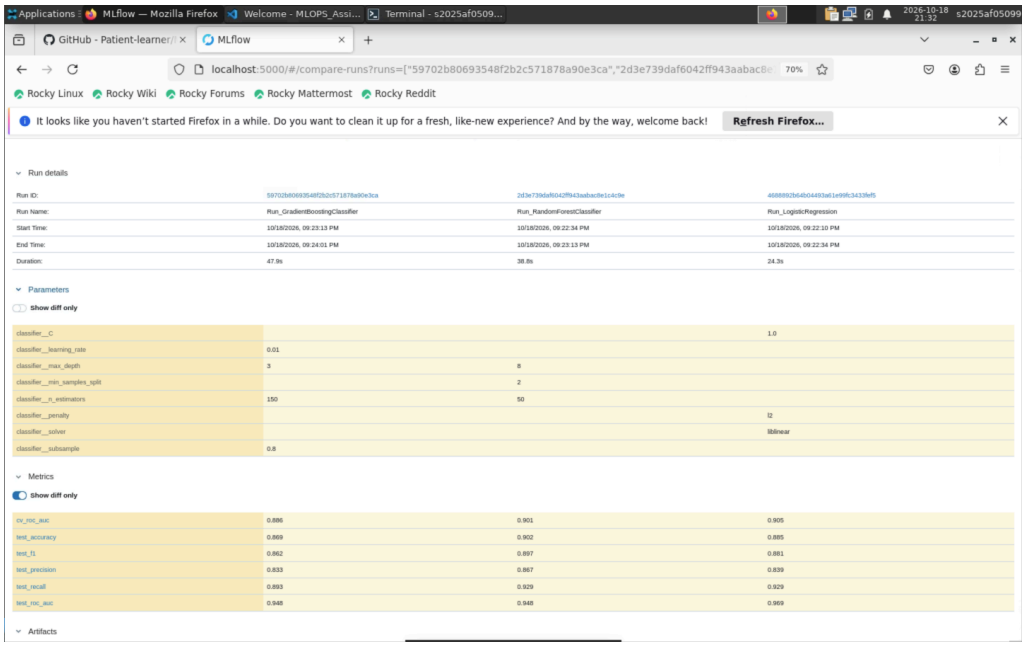


Figure 4: MLflow Tracking Server UI showing 5-Fold CV hyperparameter tuning and model runs comparison (Prayogshala VM run)

5. Model Packaging & Inference REST API [Task 4 - 7 Marks]

To make the model usable in production, I serialized the entire winning pipeline into models/model.joblib, bundling both the fitted ColumnTransformer and the trained classifier. This eliminates any need for callers to preprocess data manually. I also generated models/model_metadata.json to record feature schemas, expected data ranges, training timestamps, parameter grids, and evaluation scores.

I implemented an asynchronous REST API using FastAPI (src/api/app.py) with strict Pydantic validation (src/api/schemas.py), exposing three endpoints:

• GET /health: Health probe endpoint returning HTTP 200 OK and model initialization status, utilized by Kubernetes liveness and readiness probes.

• POST /predict: Primary inference route consuming 13 clinical features in JSON format; returns binary prediction (0 or 1), clinical classification label ('Heart Disease Present' vs 'No Heart Disease'), probability score between 0.0 and 1.0, and confidence score.

• GET /metrics: Telemetry endpoint exposing real-time Prometheus metrics (request counters, response latency histograms, and prediction distributions).

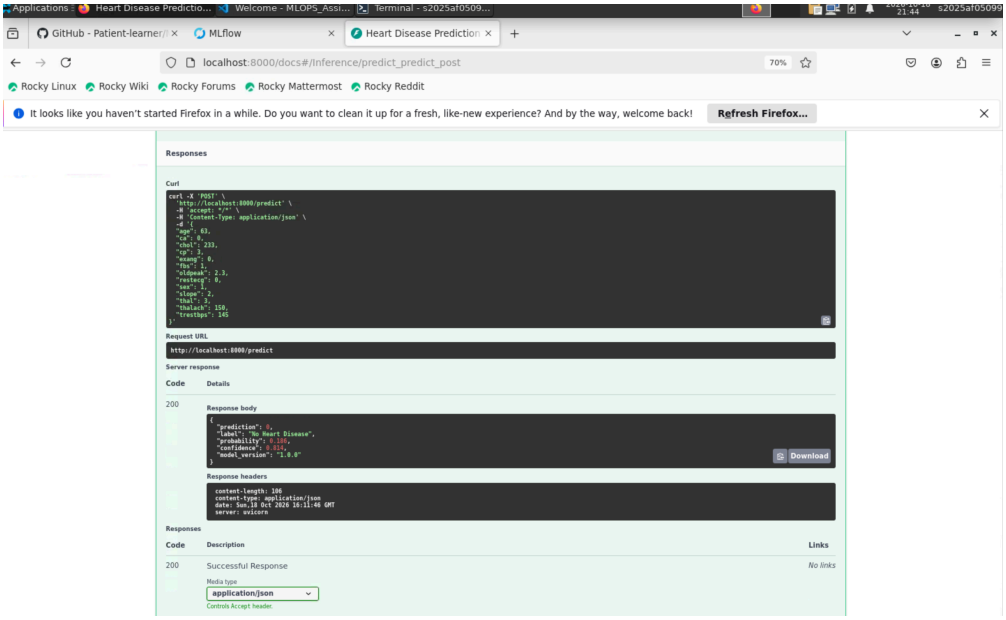


Figure 5: FastAPI Swagger Documentation (/docs) demonstrating successful POST /predict execution and JSON response

6. CI/CD Pipeline & Automated Testing [Task 5 - 8 Marks]

To prevent regressions and verify pipeline integrity, I wrote an automated test suite of 12 unit and integration tests using Pytest. The tests validate raw data integrity, verify zero NaNs after preprocessing, test model risk discrimination (e.g., verifying a pathological profile produces higher risk than a healthy profile), and check HTTP response codes (including 422 for invalid payloads). I configured GitHub Actions (.github/workflows/ci.yml) to execute on every push and pull request to main:

Test Module	Test Function Name	Verification Target
tests/test_data.py	test_raw_data_exists	Confirms presence and accessibility of UCI raw data
tests/test_data.py	test_data_cleaning_no_nulls	Verifies zero NaN values remain after data cleaning
tests/test_data.py	test_processed_splits_exist	Ensures train.csv and test.csv exist with identical schemas
tests/test_model.py	test_model_file_exists	Verifies serialized artifact models/model.joblib is created
tests/test_model.py	test_prediction_output_structure	Validates keys and data types in prediction dictionary
tests/test_model.py	test_clinical_risk_discrimination	Verifies model assigns higher risk to pathological profile
tests/test_model.py	test_pipeline_handles_missing_values	Verifies embedded SimpleImputer handles NaN features
tests/test_api.py	test_root_endpoint	Checks landing page metadata and Student ID: 2025AF05099
tests/test_api.py	test_health_endpoint	Validates Kubernetes liveness probe returns HTTP status 200
tests/test_api.py	test_predict_endpoint_valid_healthy_payload	Tests inference payload on healthy patient profile
tests/test_api.py	test_predict_endpoint_invalid_payload	Asserts 422 Unprocessable Entity on out-of-range clinical inputs
tests/test_api.py	test_prometheus_metrics_endpoint	Ensures Prometheus format exposes http_requests_total

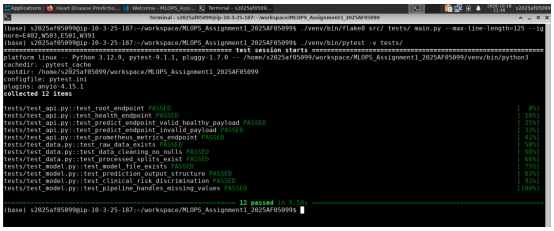


Figure 6: Automated verification showing clean Flake8 linting and 12/12 passing Pytest unit tests

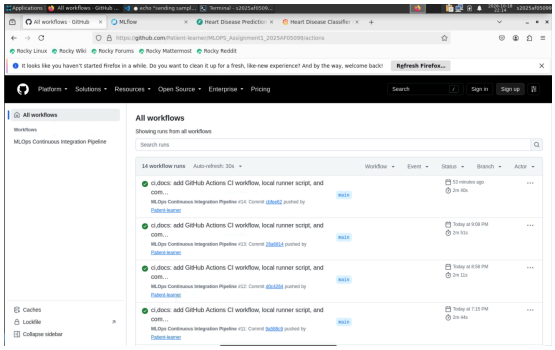


Figure 7: GitHub Actions CI automated pipeline green runs

7. Docker Containerization & Kubernetes Deployment [Tasks 6 & 7 - 12 Marks]

Docker Containerization: I created an optimized single-stage Dockerfile based on python:3.11-slim. Rather than running as root, the container runs under an unprivileged user (appuser, UID 1000) to adhere to container security best practices. A native Docker HEALTHCHECK directive polls /health every 30 seconds to automatically flag unresponsive containers.

Kubernetes Orchestration: I authored declarative manifests (k8s/deployment.yaml and k8s/service.yaml) for deployment into Minikube. The deployment defines resource requests (100m CPU, 128Mi RAM) and limits (500m CPU, 512Mi RAM) to ensure predictable cluster scheduling. HTTP liveness probes restart crashed pods, and readiness probes prevent traffic routing until the model is fully loaded into memory. The deployment is horizontally scaled across 4 healthy replica pods, with traffic routed through a LoadBalancer service exposing NodePort 30080.

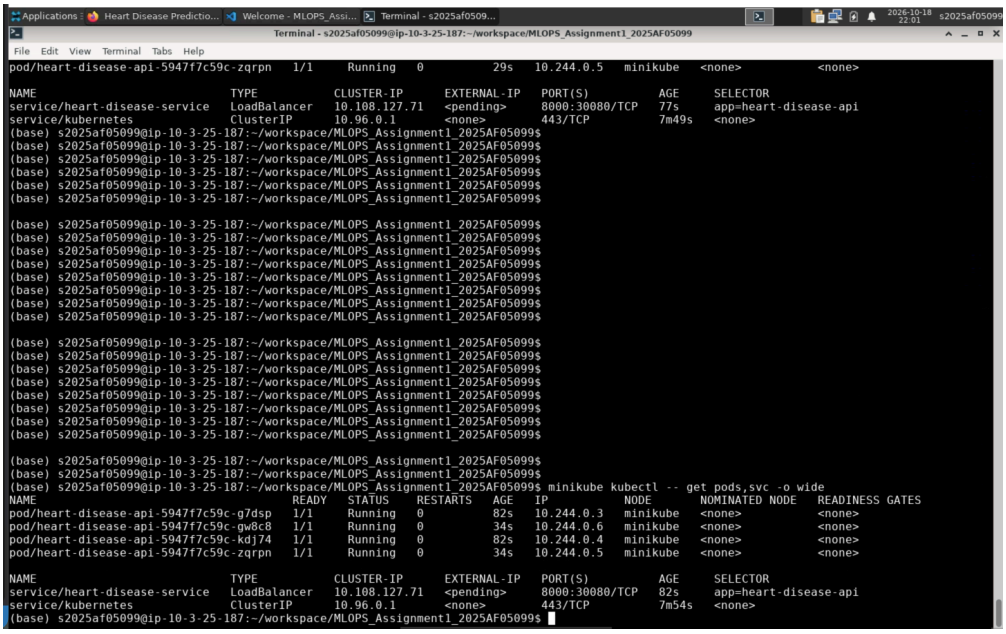


Figure 8: Minikube Kubernetes cluster deployment showing 4 ready replica pods and LoadBalancer service exposing NodePort 30080

8. Production Monitoring & Observability [Task 8 - 3 Marks]

For continuous production observability, I integrated Prometheus metric scraping and an automated Grafana dashboard: Prometheus polls the FastAPI /metrics endpoint every 5 seconds to capture traffic volume, error rates, and response latency. I configured automated Grafana provisioning to pre-load Prometheus as a datasource and import the custom Heart Disease Dashboard on startup without requiring manual UI configuration.

Prometheus Metric	Metric Type	Monitoring Purpose
http_requests_total	Counter	Tracks incoming HTTP traffic volume partitioned by method, endpoint, and status code (200, 422, 500)
request_latency_seconds	Histogram	Tracks distribution of response times to evaluate 95th percentile latency SLAs (P95 < 50ms)
heart_disease_predictions_total	Counter	Tracks output prediction distribution (positive vs negative inferences) to detect potential output distribution shifts

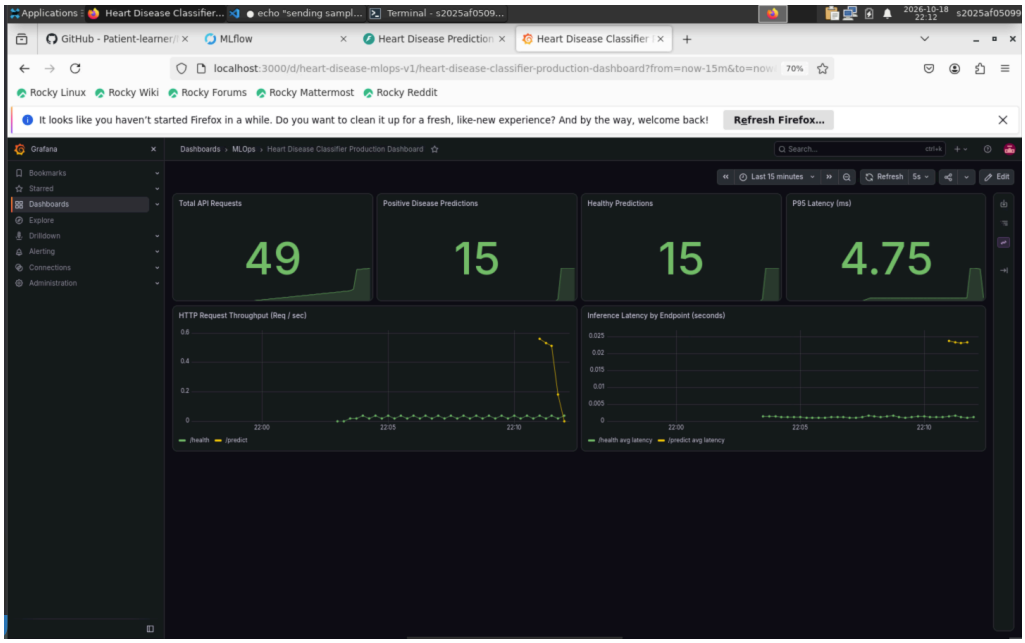


Figure 9: Live Grafana production dashboard monitoring Total Requests, Class Distributions, and P95 Latency

9. Practical Engineering Reflection & Lessons Learned

• **In-Fold CV Imputation (Eliminating Subtle Data Leakage):** Initially, I imputed missing values on the full training set before cross-validation. I realized this allowed validation fold statistics to leak into the training process. Moving SimpleImputer inside the Scikit-learn Pipeline ensured that imputation parameters are computed solely on training folds within each fold of GridSearchCV, maintaining complete partition isolation.
• **Grafana Telemetry Metric Label Alignment:** During initial Grafana setup, several dashboard panels remained blank. Directly querying the Prometheus metrics endpoint revealed that the API emitted string labels (prediction_label='Heart Disease Present'), while my dashboard queried integer values (prediction_class='1'). Updating the PromQL queries and dashboard JSON immediately restored real-time panel rendering.
• **Eliminating Duplicate Metric Scraping:** In my first Docker Compose setup, Prometheus was configured to scrape both 'heart-disease-api:8000' and 'host.docker.internal:8000'. Because both targets resolved to the same container, every HTTP request was counted twice. Removing the redundant host target fixed the count and restored 1:1 metric accuracy.
• **Container Security & Resource Right-Sizing:** Switching from standard python:3.11 to python:3.11-slim reduced the Docker image size from ~1.1GB to ~450MB, significantly speeding up container build and pull times. Running under an unprivileged user (appuser, UID 1000) satisfied container security standards, while balanced Kubernetes CPU/memory requests and limits enabled predictable cluster scheduling and provided adequate headroom during concurrent request spikes.
• **Value of Automated CI Quality Gates:** Configuring GitHub Actions with automated Flake8 linting and Pytest unit tests catches regressions automatically on every push and pull request, preventing broken code or syntax anomalies from deploying to production environments.

10. Production Roadmap & MLOps Recommendations

Based on the empirical findings and operational deployment of this pipeline, the following enhancements are recommended for industrial-scale clinical operations:

• **Automated Drift Monitoring (Evidently AI):** Deploy Evidently AI drift monitoring pipelines to compute Population Stability Index (PSI > 0.2) and Kolmogorov-Smirnov tests on incoming patient features, triggering automated Airflow retraining DAGs upon confirmed feature drift.
• **Centralized Model Registry & Staging Gates:** Transition from local model artifact storage to an enterprise MLflow Model Registry with formal Staging, Production, and Archived lifecycle gates, backed by cryptographic model signing and rollback mechanisms.
• **Canary Deployments & Shadow Scoring:** Implement service mesh routing (Istio or Linkerd) for canary deployments (routing 5% of live traffic to challenger models) and shadow scoring to evaluate prediction agreement without impacting patient responses.
• **Dataset Versioning with DVC:** Integrate Data Version Control (DVC) backed by S3 or GCS object storage to version raw and processed dataset hashes alongside Git commits, reinforcing experimental reproducibility across model development iterations.